Übungsblatt 5

CSS-Layouts, Responsive Design, DOM-Manipulation

5430UE Web and Data Engineering

13. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Beherrschung zentraler CSS-Konzepte wie Selektoren, Spezifität, Box-Modell und Normal Flow
- Entwicklung responsiver Benutzeroberflächen mithilfe von Flexbox, CSS Grid und Media Queries
- Analyse und Umsetzung adaptiver Layoutstrategien für unterschiedliche Endgeräte
- Implementierung interaktiver Webanwendungen mittels DOM-Manipulation und Event-Handling



API Gateway und Fat Clients

API Gateway und Fat Clients

Wir betrachten folgendes Szenario:
Ein E-Commerce-Unternehmen hat seine Anwendung in 8 Microservices aufgeteilt. Die mobile App muss für eine einzige Produktseite Daten von 4 verschiedenen Services abrufen.

1. Erklären Sie kurz, was ein **API Gateway** ist. Welches Problem löst es in diesem Szenario?
2. Erklären Sie kurz, worin sich ein **API Gateway** von einem einfachen **API Proxy (Reverse Proxy)** unterscheidet.
3. Diskutieren Sie **Fat Client vs. Thin Client**: Nennen Sie je zwei Vor- und Nachteile eines Fat Clients für Web-Anwendungen.

API Gateway und Fat Clients — Lösung (1/4)

Zu 1)

API Gateway und Fat Clients — Lösung (1/4)

Zu 1)

Ein API-Gateway ist eine Komponente, die als Vermittler zwischen Clients und den einzelnen Microservices fungiert. Es stellt eine zentrale Schnittstelle für externe Client-Anfragen bereit, nimmt diese entgegen und leitet sie intern an die jeweils zuständigen Microservices weiter. Dabei kann es Antworten bündeln und dem Client die Kenntnis der internen Service-Topologie ersparen, wodurch die Komplexität der Client-Server-Kommunikation reduziert wird. Das Gateway kann auch verschiedene Funktionen ausführen, wie Authentifizierung, Autorisierung, Load Balancing, Caching und Protokollierung.

API Gateway und Fat Clients — Lösung (1/4)

Zu 1)

Ein API-Gateway ist eine Komponente, die als Vermittler zwischen Clients und den einzelnen Microservices fungiert. Es stellt eine zentrale Schnittstelle für externe Client-Anfragen bereit, nimmt diese entgegen und leitet sie intern an die jeweils zuständigen Microservices weiter. Dabei kann es Antworten bündeln und dem Client die Kenntnis der internen Service-Topologie ersparen, wodurch die Komplexität der Client-Server-Kommunikation reduziert wird. Das Gateway kann auch verschiedene Funktionen ausführen, wie Authentifizierung, Autorisierung, Load Balancing, Caching und Protokollierung.

Lösung:
Im gegebenen Szenario muss die mobile App für eine Produktseite Daten von vier verschiedenen Microservices abrufen. Ohne API-Gateway müsste der Client diese Services direkt ansprechen und die Antworten selbst koordinieren (s. Fat Client). Mit einem API-Gateway sendet die App nur eine Anfrage an das Gateway, das die vier Service-Aufrufe intern orchestriert, die Antworten aggregiert und als einheitliche Antwort zurückliefert.

API Gateway und Fat Clients — Lösung (2/4)

Zu 2)

Ein API-Proxy ist eine ähnliche Komponente wie das API-Gateway, die auch als Vermittler zwischen Clients und den einzelnen Services agiert. Der Hauptunterschied zwischen einem API-Gateway und einem API-Proxy besteht darin, dass das API-Gateway zusätzliche Funktionen bietet, wie z.B. Bündelung, Transformation und Aggregation von Anfragen, Rate Limiting, Authentifizierung, Autorisierung, Load Balancing, Response-Caching und Protokollierung. Ein API-Proxy dient hingegen hauptsächlich als Weiterleitung von Anfragen an den entsprechenden Service und bietet keine zusätzlichen Funktionen.

API Gateway und Fat Clients — Lösung (3/4)

Zu 3)

Zusatz:

API Gateway und Fat Clients — Lösung (3/4)

Zu 3)

Zusatz:

Fat Client:
Ein Fat Client ist eine Anwendungsarchitektur, bei der ein Großteil der Anwendungslogik, Datenverarbeitung und Benutzerinteraktion auf dem Client selbst ausgeführt wird. Der Server übernimmt hauptsächlich die Bereitstellung und Speicherung von Daten.

API Gateway und Fat Clients — Lösung (3/4)

Zu 3)

Zusatz:

Fat Client:
Ein Fat Client ist eine Anwendungsarchitektur, bei der ein Großteil der Anwendungslogik, Datenverarbeitung und Benutzerinteraktion auf dem Client selbst ausgeführt wird. Der Server übernimmt hauptsächlich die Bereitstellung und Speicherung von Daten.

Thin Client:
Ein Thin Client ist eine Anwendungsarchitektur, bei der die wesentliche Anwendungslogik und Verarbeitung auf dem Server stattfindet. Der Server erzeugt die vollständigen HTML-Seiten, die vom Browser lediglich gerendert werden. Der Client enthält nur wenig eigene Logik; jede Benutzerinteraktion führt in der Regel zu einem neuen HTTP-Request und einem vollständigen Seiten-Reload.



API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

Nachteile:

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung

Nachteile:

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden

Nachteile:

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden
- Weniger Netzwerkverkehr: Durch die Verarbeitung von Daten auf dem Client kann der Netzwerkverkehr reduziert werden, da weniger Daten zwischen Client und Server übertragen werden müssen.

Nachteile:

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden
- Weniger Netzwerkverkehr: Durch die Verarbeitung von Daten auf dem Client kann der Netzwerkverkehr reduziert werden, da weniger Daten zwischen Client und Server übertragen werden müssen.

Nachteile:

- Mehr JavaScript-Code, was zu längeren Ladezeiten beim ersten Aufruf führt

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden
- Weniger Netzwerkverkehr: Durch die Verarbeitung von Daten auf dem Client kann der Netzwerkverkehr reduziert werden, da weniger Daten zwischen Client und Server übertragen werden müssen.

Nachteile:

- Mehr JavaScript-Code, was zu längeren Ladezeiten beim ersten Aufruf führt
- Sicherheitsbedenken: Sicherheitskritische Logik darf nie beim Client liegen. Client ist anfälliger für Angriffe

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden
- Weniger Netzwerkverkehr: Durch die Verarbeitung von Daten auf dem Client kann der Netzwerkverkehr reduziert werden, da weniger Daten zwischen Client und Server übertragen werden müssen.

Nachteile:

- Mehr JavaScript-Code, was zu längeren Ladezeiten beim ersten Aufruf führt
- Sicherheitsbedenken: Sicherheitskritische Logik darf nie beim Client liegen. Client ist anfälliger für Angriffe

API Gateway und Fat Clients — Lösung (4/4)

Vorteile:

- Reaktionsschnelle UI (Logik im Browser), reduziert Serverlast, bessere Benutzererfahrung
- Offline-Fähigkeit möglich: Fat Client führt die meisten oder alle Verarbeitungs- und Datenmanipulationsaufgaben auf dem Client aus. Damit kann die Anwendung auch ohne Internetverbindung ausgeführt werden
- Weniger Netzwerkverkehr: Durch die Verarbeitung von Daten auf dem Client kann der Netzwerkverkehr reduziert werden, da weniger Daten zwischen Client und Server übertragen werden müssen.

Nachteile:

- Mehr JavaScript-Code, was zu längeren Ladezeiten beim ersten Aufruf führt
- Sicherheitsbedenken: Sicherheitskritische Logik darf nie beim Client liegen. Client ist anfälliger für Angriffe

CSS Spezifizität

CSS Spezifizität (1/3)

Gegeben sind der folgende HTML-Code und das zugehörige CSS-Stylesheet:

index.html

```
<!DOCTYPE html>
<html lang="de">
   <head>
      <meta charset="UTF-8">
      <title>Spezifizitaet</title>
      <link rel="stylesheet" href="style.css">
    </head>
   <body id="body-main">
      <div id="container">
          <span id="item-1" class="highlight important">Ziel-Text</span>
      </div>
   </body>
</html>
```

CSS Spezifizität (2/3)

style.css

```css
/* 1 */
#body-main div.highlight { color: black; }

/* 2 */
#item-1 { color: blue; }

/* 3 */
span { color: grey; background-color: white; }

/* 4 */
body #container .important { color: cyan; }

/* 5 */
div #item-1 { color: green; }

/* 6 */
.highlight { color: red; }

/* 7 */
```



CSS Spezifizität (3/3)

1. Berechnen Sie für jeden der 8 Selektoren die Spezifität nach dem Schema *(ID, Klasse, Element)*.
2. Ordnen Sie die Selektoren in aufsteigender Priorität (niedrigste zuerst).
3. Welches visuelle Erscheinungsbild ergibt sich für das span-Element bezüglich Textfarbe (`color`) und Hintergrundfarbe (`background-color`)?

CSS Spezifizität — Lösung (1/2)

Nr.	Selektor	Spezifizität	Priorität (1 niedrigst)
1	`#body-main div.highlight`	(1, 1, 1)	7
2	`#item-1`	(1, 0, 0)	4
3	`span`	(0, 0, 1)	1
4	`body #container .important`	(1, 1, 1)	8
5	`div #item-1`	(1, 0, 1)	5
6	`.highlight`	(0, 1, 0)	2
7	`#body-main span`	(1, 0, 1)	6
8	`div span.important`	(0, 1, 2)	3

CSS Spezifizität — Lösung (1/2)

Erscheinungbild:

- Textfarbe: `cyan` (durch Selektor 4, da höchste Priorität).
- `yellow` (Selektor 8 ist die spezifischste Regel, die eine Hintergrundfarbe definiert; die Angabe aus Selektor 3 wird überschrieben).

Erläuterung:
`A.B`: Selektiert Elemente des Typs A, die zugleich der Klasse B angehören.
`A .B`: Selektiert alle Elemente der Klasse B, die Nachfahren eines Elements vom Typ A sind.

Normal Flow und Element-Typen

Normal Flow und Element-Typen

1. Beschreiben Sie das Standardverhalten (Normal Flow) von *Block-Elementen* und *Inline-Elementen* im Browser, wenn keine besonderen Layoutregeln wie Flexbox oder Grid angewendet werden.

2. Gegeben sei der folgende HTML-Code:

```
<p>Ein Absatz mit einem <span>Inline-Text</span>.</p>
```

Was passiert visuell, wenn auf das -Element im CSS die Regel `display: block;` angewendet wird? Ändert sich dadurch die semantische Bedeutung des Elements?

Normal Flow und Element-Typen — Lösung

Zu 1)

Normal Flow und Element-Typen — Lösung

Zu 1)

- **Block-Elemente:** Beginnen typischerweise in einer neuen Zeile und nehmen die gesamte verfügbare Breite des Elternelements ein (z. B. `<div>`, `<p>`).

Normal Flow und Element-Typen — Lösung

Zu 1)

- **Block-Elemente:** Beginnen typischerweise in einer neuen Zeile und nehmen die gesamte verfügbare Breite des Elternelements ein (z. B. `<div>`, `<p>`).
- **Inline-Elemente:** Bleiben im laufenden Textfluss und belegen nur so viel Platz, wie ihr tatsächlicher Inhalt benötigt (z. B.).

Normal Flow und Element-Typen — Lösung

Zu 1)

- **Block-Elemente:** Beginnen typischerweise in einer neuen Zeile und nehmen die gesamte verfügbare Breite des Elternelements ein (z. B. `<div>`, `<p>`).
- **Inline-Elemente:** Bleiben im laufenden Textfluss und belegen nur so viel Platz, wie ihr tatsächlicher Inhalt benötigt (z. B.).

Zu 2)

Visuell führt `display: block;` dazu, dass der Textteil „Inline-Text" in eine neue Zeile umbricht und den restlichen verfügbaren Platz in der Breite einnimmt. Der Text vor und nach dem steht dann jeweils in einer eigenen Zeile.

Normal Flow und Element-Typen — Lösung

Zu 1)

- **Block-Elemente:** Beginnen typischerweise in einer neuen Zeile und nehmen die gesamte verfügbare Breite des Elternelements ein (z. B. `<div>`, `<p>`).
- **Inline-Elemente:** Bleiben im laufenden Textfluss und belegen nur so viel Platz, wie ihr tatsächlicher Inhalt benötigt (z. B.).

Zu 2)

Visuell führt `display: block;` dazu, dass der Textteil „Inline-Text" in eine neue Zeile umbricht und den restlichen verfügbaren Platz in der Breite einnimmt. Der Text vor und nach dem steht dann jeweils in einer eigenen Zeile.

Semantik: Die semantische Bedeutung des HTML-Elements ändert sich dadurch **nicht**. CSS beeinflusst ausschließlich die visuelle Darstellung (Präsentation), nicht aber die Bedeutung des Codes für den Browser oder Screenreader.

Box-Modell und Selektoren

Box-Modell und Selektoren

Gegeben folgendes CSS:

```css
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 16px;
}
```

1. Wie breit ist `.card` im Standard-Box-Modell (`content-box`)? Begründen Sie.
2. Wie breit wäre `.card` mit `box-sizing: border-box`?
3. Schreiben Sie einen CSS-Selektor, der nur das erste `.card`-Element innerhalb eines `<section>` trifft.

Box-Modell und Selektoren — Lösung

Box-Modell und Selektoren — Lösung

Zu 1) Standard (content-box): `width` gilt nur für den Content-Bereich. Padding und Border kommen zusätzlich dazu.
`Gesamtbreite = 300 + 20 + 20 + 2 + 2 = 344`px.
Hinweis: Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.

Box-Modell und Selektoren — Lösung

Zu 1) Standard (content-box): `width` gilt nur für den Content-Bereich. Padding und Border kommen zusätzlich dazu.
`Gesamtbreite = 300 + 20 + 20 + 2 + 2 = 344px`.
Hinweis: Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.

Zu 2) box-sizing: border-box: `width:300px` ist das Gesamtmaß inklusive Padding und Border.
`Content-Breite = 300 - 40 - 4 = 256px`.
Die Gesamtbreite des Elements (Content, Padding und Border) bleibt `300px`.

Box-Modell und Selektoren — Lösung

Zu 1) Standard (content-box): `width` gilt nur für den Content-Bereich. Padding und Border kommen zusätzlich dazu.
`Gesamtbreite = 300 + 20 + 20 + 2 + 2 = 344px`.
Hinweis: Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.

Zu 2) box-sizing: border-box: `width:300px` ist das Gesamtmaß inklusive Padding und Border.
`Content-Breite = 300 - 40 - 4 = 256px`.
Die Gesamtbreite des Elements (Content, Padding und Border) bleibt `300px`.

Zu 3) Selektor:

Box-Modell und Selektoren — Lösung

Zu 1) Standard (content-box): `width` gilt nur für den Content-Bereich. Padding und Border kommen zusätzlich dazu.
`Gesamtbreite = 300 + 20 + 20 + 2 + 2 = 344px`.
Hinweis: Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.

Zu 2) box-sizing: border-box: `width:300px` ist das Gesamtmaß inklusive Padding und Border.
`Content-Breite = 300 - 40 - 4 = 256px`.
Die Gesamtbreite des Elements (Content, Padding und Border) bleibt `300px`.

Zu 3) Selektor:

```
section .card:first-child { }

/* oder praeziser: */

section > .card:first-of-type { }
```

Responsive Layout mit Flexbox

Responsive Layout mit Flexbox

Implementieren Sie ein responsives Layout, das auf Desktop-Monitoren (≥ 768px) drei Spalten nebeneinander anzeigt und auf Mobilgeräten (< 768px) auf eine einspaltige Ansicht umspringt. Nutzen Sie dafür ausschließlich Flexbox-Eigenschaften für den Container `.product-layout` und die Kindelemente `.product-card`.

Ein entsprechendes Projekt finden Sie in Stud.IP unter dem Namen *flex_layout*. Ergänzen Sie dort die fehlenden Deklarationen in der Datei `style.css` an den markierten Stellen.

Hinweis:
Ein sehr gutes Cheat-Sheet zu Flexbox finden Sie unter: https://css-tricks.com/snippets/css/a-guide-to-flexbox/

Responsive Layout mit Flexbox — Lösung (1/3)

```css
/* Container-Styling */
.product-layout {
  display: flex;
  flex-wrap: wrap;       /* Erlaubt Umbruch in neue Zeilen */
  gap: 1rem;             /* Abstand zwischen den Karten */
}

/* Karten-Styling (Flex-Items) */
.product-card {
  /* Flex-Eigenschaften (Desktop-Standard: 3 Spalten) */
  flex-grow: 1;           /* Karten fuellen den Platz aus */
  flex-shrink: 1;         /* Karten verkleinern sich bei Platzmangel */
  flex-basis: calc(33.333% - 1rem); /* Basisgroesse fuer 3 Spalten minus Gap */

  min-width: 0;           /* Verhindert Layout-Fehler bei langem Inhalt */
}

/* Mobile Anpassung (Viewports kleiner als 768px) */
@media (max-width: 767px) {
```

Responsive Layout mit Flexbox — Lösung (2/3)

Zusatz: Idee von Flexbox

Flexbox richtet die Elemente entlang einer Achse aus. Mittels des Attributs `flex-direction` des Containers wird diese Achse ausgewählt (Default: `row`, Weitere Werte: `row-reverse`, `column`,...)

Zusatz: Erläuterungen einzelner Container-Attribute

- `display: flex;`: Aktiviert den Flexbox-Kontext für den Container, wodurch dessen direkte Kindelemente zu flexiblen Elementen werden.
- `flex-wrap: wrap | wrap-reverse | nowrap;`: Bestimmt, ob die Flex-Elemente gezwungen werden, in einer einzigen Zeile zu bleiben, oder bei Platzmangel in eine neue Zeile umbrechen dürfen.
- `flex-direction: row | row-reverse | column | column-reverse;`: Bestimmt die Hauptachse des Flex-Containers und legt damit fest, ob die Flex-Elemente horizontal in einer Zeile oder vertikal in einer Spalte angeordnet werden sowie in welcher Reihenfolge sie erscheinen.
- `justify-content: flex-start | flex-end | center |...;`: Bestimmt die Ausrichtung der Flex-Elemente entlang der Hauptachse des Flex-Containers und legt fest, wie der verfügbare freie Raum zwischen und um die Elemente verteilt wird.



Responsive Layout mit Flexbox — Lösung (3/3)

Zusatz: Erläuterungen einzelner Items-Attribute

- `flex-grow`: Legt den Faktor fest, mit dem ein Element im Verhältnis zu seinen Nachbarn wächst, um den verbleibenden freien Raum im Container zu füllen.
- `flex-shrink`: Definiert die Fähigkeit eines Elements, seine Größe zu verringern, wenn der Platz im Container nicht mehr für alle Elemente ausreicht.
- `flex-basis`: Gibt die initiale Standardgröße eines Elements an, bevor der verbleibende Platz durch die Wachstums- oder Schrumpffaktoren verteilt wird.
- `flex` (Kurzschreibweise): Fasst die drei Eigenschaften flex-grow, flex-shrink und flex-basis in einer einzigen Deklaration zusammen, um den Code übersichtlicher und weniger fehleranfällig zu gestalten. (Default: `flex: 0 1 auto`)
- `min-width: 0`: Hebt die standardmäßige Mindestbreite von Flex-Items auf und erlaubt dadurch, dass sich ein Element auch kleiner als sein Inhalt zusammenziehen kann, wodurch verhindert wird, dass lange Inhalte (z. B. Wörter oder Bilder) das Flex-Layout über seine berechnete Breite hinaus aufweiten.

CSS Grid Layout

CSS Grid Layout (1/3)

1. Erläutern Sie anhand einer Faustregel, wann der Einsatz von CSS Grid und wann der Einsatz von Flexbox für ein Layout sinnvoll ist. Beziehen Sie sich in Ihrer Antwort auch auf das Verhalten der `.product-card`-Elemente aus der vorherigen Flexbox-Aufgabe.
2. Erklären Sie kurz die Funktionsweise der CSS-Funktion `minmax()` in Kombination mit `auto-fit` für ein responsives Grid-Layout. Welcher Vorteil ergibt sich hierdurch im Vergleich zur Verwendung von starren Breakpoints (Media Queries)?

CSS Grid Layout (2/3)

3. **Explizites Raster (mit Media Query):** Gegeben ist der Container `.product-detail`, der ein Bild (`image`), Kurzinfos (`info`) und eine ausführliche Beschreibung (`desc`) enthält. Implementieren Sie das CSS so, dass auf Desktops folgendes 2-spaltiges Raster entsteht:

   - Spalte 1 (`1fr`) enthält oben das Bild.
   - Spalte 2 (`1.2fr`) enthält die Kurzinfos.
   - Die Beschreibung (`desc`) soll in einer zweiten Zeile stehen und **beide Spalten** überspannen.

   Auf mobilen Geräten (`max-width: 700px`) sollen alle drei Elemente einfach untereinander stehen. Nutzen Sie für die Positionierung ausschließlich `grid-template-areas`.

   Ein entsprechendes Projekt mit der Dateistruktur für die Aufgaben c) und d) finden Sie in Stud.IP unter dem Namen *grid_layout*. Ergänzen Sie dort die fehlenden Deklarationen in der Datei `style.css` an den markierten Stellen.

CSS Grid Layout (3/3)

4. **Responsives Raster (ohne Media Query):** Entwickeln Sie nun ein responsives Grid für den Container `.product-grid`, das seine Spaltenanzahl automatisch an den verfügbaren Platz anpasst, *ohne* Media Queries zu verwenden. Die Spalten sollen eine Mindestbreite von `250px` haben und sich ansonsten gleichmäßig ausdehnen. Stellen Sie durch eine weitere Grid-Eigenschaft sicher, dass alle automatisch erzeugten Zeilen mindestens `150px` hoch sind, sich bei viel Inhalt aber automatisch vergrößern.

CSS Grid Layout — Lösung (1/4)

CSS Grid Layout — Lösung (1/4)

Zu 1) Faustregel: Flexbox eignet sich am besten, wenn der Inhalt dynamisch verteilt wird und die Reihenfolge weitgehend linear bleibt. Bei der vorherigen Aufgabe wurden die `.product-card`-Elemente flexibel entlang der Hauptachse verteilt und sind bei Platzmangel einfach umgebrochen (`flex-wrap`). Grid ist hingegen die richtige Wahl, wenn das Layout als explizites 2D-Raster (Zeilen und Spalten gemeinsam) vorgegeben wird und die Struktur fest im Vordergrund steht.

CSS Grid Layout — Lösung (1/4)

Zu 1) Faustregel: Flexbox eignet sich am besten, wenn der Inhalt dynamisch verteilt wird und die Reihenfolge weitgehend linear bleibt. Bei der vorherigen Aufgabe wurden die `.product-card`-Elemente flexibel entlang der Hauptachse verteilt und sind bei Platzmangel einfach umgebrochen (`flex-wrap`). Grid ist hingegen die richtige Wahl, wenn das Layout als explizites 2D-Raster (Zeilen und Spalten gemeinsam) vorgegeben wird und die Struktur fest im Vordergrund steht.

Zu 2) `auto-fit` erzeugt automatisch so viele Spalten, wie in den Container passen.
`minmax()` sorgt dafür, dass eine Spalte einen Mindestwert (z. B. `250px`) nicht unterschreitet, bei mehr Platz aber flexibel mitwächst (z. B. `1fr`).
Vorteil: Der Browser berechnet die Spaltenanzahl dynamisch aus dem verfügbaren Platz. Ein responsives Umbrechen geschieht fließend, ohne dass starre Breakpoints für „2 Spalten" oder „3 Spalten" via Media Queries definiert werden müssen.

CSS Grid Layout — Lösung (2/4)

Zu 3) Explizites Raster mit Umbruch via Media Query:

```
/* Zuweisung der Bezeichner */
.product-image { grid-area: image; }
.product-info  { grid-area: info; }
.product-desc  {
  grid-area: desc;
  background-color: #f0f0f0; /* Optische Abhebung */
}

.product-detail {
  display: grid;
  gap: 1.5rem;
  /* Desktop-Layout:
  Zeile 1: Bild und Info nebeneinander
  Zeile 2: Beschreibung ueber beide Spalten hinweg */
  grid-template-areas:
  "image info"
  "desc  desc";
  grid-template-columns: 1fr 1.2fr;
}
```

CSS Grid Layout — Lösung (3/4)

Zusatz: Warum Grid statt Flexbox?

Grid ermöglicht dieses Layout mit „flachem" HTML, während Flexbox für die Desktop-Ansicht einen zusätzlichen Wrapper um Bild und Info bräuchte, um diese von der Beschreibung abzugrenzen. Mit Grid lässt sich die Beschreibung via grid-template-areas ganz einfach über beide Spalten strecken, ohne die HTML-Struktur für verschiedene Bildschirmgrößen ändern zu müssen.

CSS Grid Layout — Lösung (4/4)

Zu 4) Responsives Raster und implizite Zeilen:

```css
.product-grid {
  display: grid;
  gap: 1rem;
  /* Automatische Spaltenanzahl ohne Media Query */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  /* Zusatz: Hoehe der impliziten Zeilen steuern */
  grid-auto-rows: minmax(150px, auto);
}
```

DOM-Manipulation und Events

DOM-Manipulation und Events

Implementieren Sie folgende Funktionalität in JavaScript:

Ein Button (`id: toggle-theme`) soll beim Klick die Klasse `dark-mode` auf dem `<body>` togglen (siehe `toggle`-Funktion), d.h. zur Liste der Klassen des `<body>`-Elements hinzufügen, wenn die Klasse nicht vorhanden ist und andernfalls entfernen. Zusätzlich soll der Button-Text zwischen „Dark Mode" und „Light Mode" wechseln.

Ein vorbereitetes Projekt finden Sie in Stud.IP unter dem Namen *moduswechsel*. Ergänzen Sie eine JavaScript-Datei `script.js` mit entsprechendem Code. Nutzen Sie einen EventListener, um auf Klicks des Buttons mit der ID `toggle-theme` zu reagieren.

Hinweis: Die Dateien `index.html` und `style.css` sind bereits vollständig vorgegeben und müssen nicht verändert werden.

DOM-Manipulation und Events — Lösung (1/3)

`script.js`

Version mit anonymer Pfeilfunktion (kürzer, oft für Event-Handler genutzt, kein eigenes `this`):

```
const btn = document.getElementById('toggle-theme');

btn.addEventListener('click', () => {
  document.body.classList.toggle('dark-mode');
  const isDark = document.body.classList.contains('dark-mode');

  if (isDark) {
    btn.textContent = 'Light Mode';
  } else {
    btn.textContent = 'Dark Mode'
  }
});
```

DOM-Manipulation und Events — Lösung (2/3)

Alternative mit expliziter Funktion (besser wiederverwendbar, eigenes `this`):

```
function toggleTheme() {
  document.body.classList.toggle('dark-mode');
  const isDark = document.body.classList.contains('dark-mode');

  if (isDark) {
    btn.textContent = 'Light Mode';
  } else {
    btn.textContent = 'Dark Mode';
  }
}

btn.addEventListener('click', toggleTheme);
```

DOM-Manipulation und Events — Lösung (3/3)

Zusatz: Event-Attribute vs. EventListener:
Event-Attribute (z. B. `onclick` im HTML, vorheriges Übungsblatt) binden Ereignisse direkt im HTML-Code an ein Element, vermischen jedoch Struktur und Logik und erlauben in der Regel nur einen Handler pro Event.

EventListener (z. B. `addEventListener`) trennen hingegen HTML und JavaScript sauber, ermöglichen mehrere unabhängige Handler für dasselbe Event und gelten daher als flexiblere und in der modernen Webentwicklung bevorzugte Lösung.

Materialien

Materialien zum Übungsblatt

- Aufgabe *Responsive Layout mit Flexbox*: flex_layout.zip
- Aufgabe *CSS Grid Layout*: grid_layout.zip
- Aufgabe *DOM-Manipulation und Events*: moduswechsel.zip

Lösungen:

- Aufgabe *Responsive Layout mit Flexbox*: flex_layout_lsg.zip
- Aufgabe *CSS Grid Layout*: grid_layout_lsg.zip
- Aufgabe *DOM-Manipulation und Events*: moduswechsel_lsg.zip